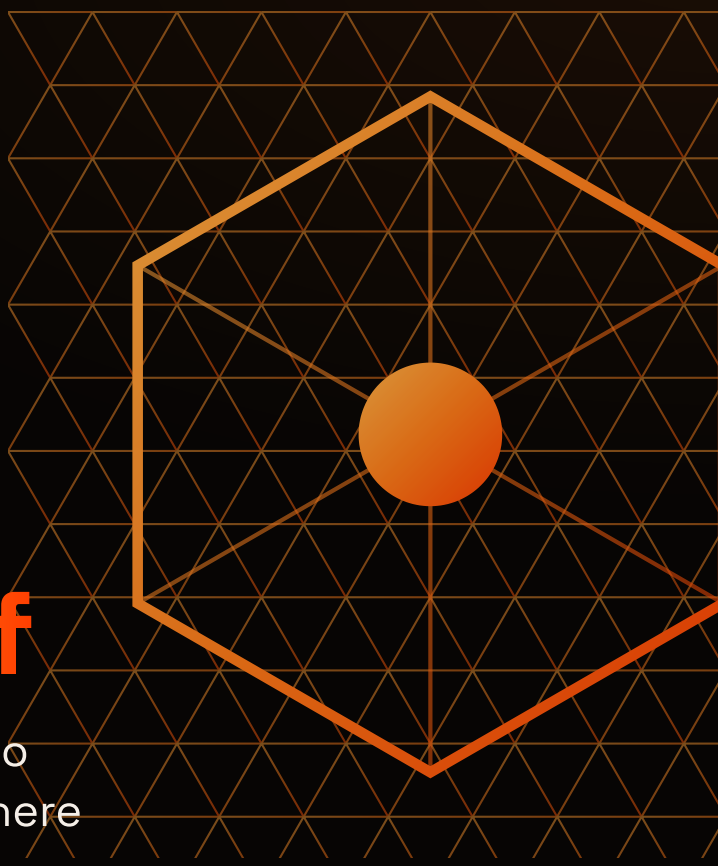
WHITEPAPER · V1.0 · OCTOBER 2026



QuantumProof

A quantum-resistant layer 1 with no elliptic-curve cryptography anywhere in the stack.

ABSTRACT

Every major blockchain authenticates value with elliptic-curve signatures that a large fault-tolerant quantum computer can break with Shor's algorithm. QuantumProof replaces them end to end. User transactions are signed with ML-DSA-44, blocks and validator attestations with ML-DSA-65 (NIST FIPS 204), and all hashing uses SHA3-256. Keys are generated client-side, every signature is verified before it reaches state, and the public testnet is live with an open explorer.

Every chain you hold signs with math a quantum computer breaks.

Public-key cryptography in today's blockchains rests on the **elliptic-curve discrete logarithm problem** (ECDLP). Bitcoin uses ECDSA and Schnorr over secp256k1, Ethereum uses ECDSA over secp256k1, and Solana and NEAR use Ed25519. Classical computers cannot solve ECDLP at these sizes. A quantum computer running **Shor's algorithm** can, in polynomial time.

NETWORK	SIGNATURE SCHEME	HARD PROBLEM	VS. SHOR
Bitcoin	ECDSA / Schnorr · secp256k1	ECDLP	Broken
Ethereum	ECDSA · secp256k1	ECDLP	Broken
Solana	Ed25519	ECDLP	Broken
NEAR	Ed25519	ECDLP	Broken
QuantumProof	ML-DSA-44 / ML-DSA-65	Module-LWE / Module-SIS	No known attack

Public keys are the attack surface

An address that has ever sent a transaction has published its public key on-chain. Given that key, Shor's algorithm recovers the private key, and with it every coin the address controls. Because ledgers are permanent and public, that exposure cannot be undone after the fact.

Harvest now, break later

An adversary does not need a quantum computer today. It only needs to record exposed public keys and signed data now and break them when the hardware arrives. Migration therefore has to be complete **before** "Q-day", not after it.

Grover's algorithm and hashing

Grover's algorithm gives a quadratic speed-up against hash pre-images, which effectively halves security bits. SHA3-256 retains roughly 128-bit security under Grover, which is why QuantumProof uses 256-bit hashes throughout.

The core problem: retrofitting post-quantum signatures onto an existing chain needs every holder to migrate keys, a hard fork, and new block-size economics. QuantumProof is designed post-quantum from genesis, so there is nothing to migrate.

Design goals and threat model

Goals

- **Zero elliptic curves.** No ECDSA, Schnorr or Ed25519 in user keys, validator keys, consensus messages or addresses.
- **Standardised primitives only.** Use NIST-published post-quantum algorithms (FIPS 204) and SHA-3 (FIPS 202) rather than novel or unreviewed schemes.
- **Client-side keys.** Secret keys are generated and used on the user's device; nodes only ever see public keys and signatures.
- **Verify everything, every time.** No signature is trusted by position or origin. Transactions are verified at admission; block and attestation signatures are verified before a block is appended.
- **Public by default.** Every block exposes its full proposer signature and attestations in the explorer, so anyone can check the cryptography byte for byte.

Threat model

ADVERSARY	CAPABILITY	MITIGATION
Quantum attacker	Runs Shor and Grover at scale	Lattice signatures (ML-DSA); 256-bit hashes
Harvesting attacker	Records all public data today	No ECC data ever published
Replay attacker	Re-broadcasts signed transactions	Per-account nonce + chain ID in signed body
Key substitution	Pairs a valid sig with another key	Address = SHA3-256(pk); node checks binding
Faulty validator	Signs an invalid or conflicting block	Attestation quorum; invalid sigs drop the block

Non-goals for testnet v1

Testnet v1 does not attempt transaction privacy, smart contracts or a permissionless validator set. These are listed on the roadmap (§7). The focus of v1 is to prove a fully post-quantum signing path from wallet to finalized block, publicly and verifiably.

Cryptographic primitives

ML-DSA (Module-Lattice Digital Signature Algorithm, formerly CRYSTALS-Dilithium) was standardised by NIST as FIPS 204 in August 2024. Its security reduces to the hardness of the **Module Learning-With-Errors** and **Module Short-Integer-Solution** problems over polynomial rings, for which no efficient classical or quantum algorithm is known.

ROLE	SCHEME	PUBLIC KEY	SIGNATURE	NIST CATEGORY
User transactions	ML-DSA-44	1,312 B	2,420 B	2
Blocks & attestations	ML-DSA-65	1,952 B	3,309 B	3
Hashes, addresses, roots	SHA3-256	—	32 B digest	~128-bit quantum
Reference: Ed25519	EdDSA	32 B	64 B	broken by Shor

Why ML-DSA

ML-DSA is NIST's primary general-purpose signature standard. It has fast verification, uses only integer arithmetic, and is constant-time friendly. **SLH-DSA** (SPHINCS+, FIPS 205) rests on hashes alone but produces 7–50 KB signatures and signs slowly. **Falcon** (FN-DSA) has smaller signatures but needs floating-point Gaussian sampling, which is hard to implement safely. ML-DSA is the best balance for a high-throughput ledger.

Measured performance

Reference node: Node.js 24 with native FIPS 204 support, single core, 64-byte message.

SCHEME	SIGN	VERIFY	KEYGEN
Ed25519 (reference)	89 µs	234 µs	87 µs
ML-DSA-44	1,504 µs	331 µs	299 µs
ML-DSA-65	2,361 µs	444 µs	492 µs

Verification, the operation every node performs for every transaction, costs within 2× of Ed25519. Signing is slower but happens once, on the sender's device.

Implementations

Nodes use the platform's native FIPS 204 implementation. Browser wallets use the open-source **@noble/post-quantum** library. Both produce standard encodings, so a signature made in the browser verifies natively on the node without translation.

Accounts and transactions

Keys and addresses

A wallet draws a 32-byte seed from the device's cryptographically secure RNG and derives an ML-DSA-44 keypair from it. The address is a hash of the public key:

```
seed  ← CSPRNG(32 bytes)
(pk, sk) ← ML-DSA-44.KeyGen(seed)
addr  = "qpf1" ‖ hex( SHA3-256(pk)[0..20] )
```

Until an account sends its first transaction, its public key is never published. Even then it is a lattice key, which gives a quantum adversary nothing to work with.

Transaction format

```
body = canonical({ amount, chain: "quantumproof-testnet-1",
                   fee, from, nonce, to })
σ    = ML-DSA-44.Sign(sk, body)
tx   = body ‖ pk (1,312 B) ‖ σ (2,420 B)
hash = SHA3-256(body ‖ σ)
```

Canonical encoding sorts keys deterministically, so the signer and every verifier hash identical bytes. The **chain** field binds the signature to one network, so a testnet signature can never be replayed on another chain.

Admission rules

- Addresses well-formed; amount positive and finite; fee equals the network fee.
- pk is exactly 1,312 bytes and SHA3-256(pk) matches **from**; this blocks key substitution.
- σ is exactly 2,420 bytes and **ML-DSA-44.Verify(pk, body, σ)** succeeds.
- nonce equals the account's next nonce, counting pending mempool entries; this blocks replays.
- The balance covers amount + fee across all pending transactions.

A transaction that fails any rule is rejected at the API with the reason, before it reaches the mempool.

Size budget

A transfer is about 3.9 KB on the wire, roughly 15–25× an Ed25519 transfer. QuantumProof treats this as a design input: block limits, gossip and storage are sized for post-quantum transactions from genesis rather than retrofitted.

Quantum-safe consensus

Testnet v1 runs a permissioned validator set of four, named **Kyber, Dilithium, Falcon and Sphincs**. Each holds an ML-DSA-65 keypair. Slots are 4 seconds, and the proposer rotates round-robin by height.

Block construction

```
proposer = V[ height mod |V| ]
txRoot   = SHA3-256( tx₁.hash ‖ … ‖ txₙ.hash )
header   = { height, prev, time, txRoot, proposer, txCount }
H        = SHA3-256( canonical(header) )
σ_p      = ML-DSA-65.Sign(sk_proposer, H)
σ_i      = ML-DSA-65.Sign(sk_i, H)   for each other validator i
```

Finality rule

$$\text{final}(B) \Leftrightarrow \text{Verify}(pk_p, H, \sigma_p) \wedge |\{\, i : \text{Verify}(pk_i, H, \sigma_i) \,\}| \geq 2$$

The proposer plus at least two of the three attesters must produce valid ML-DSA-65 signatures over the header hash, a 3-of-4 quorum. A block that fails is discarded and its transactions return to the mempool for the next slot.

Chaining and integrity

Each header commits to its parent hash and transaction root, so altering any historical transaction changes every later block hash and invalidates every signature after it. Fees in a block are credited to its proposer.

Fault behaviour

EVENT	OUTCOME
Invalid proposer signature	Block rejected; slot skipped
Fewer than 2 valid attestations	Block rejected; transactions re-queued
Invalid transaction in mempool	Never admitted (verified at the gate)
Stale nonce at execution	Transaction skipped, state untouched

Path to permissionless

The roadmap (§7) replaces the fixed set with a staked, open validator set running a BFT protocol in

Testnet, interfaces and roadmap

Live testnet: quantumproof-testnet-1

- **Explorer:** every block with its full ML-DSA-65 signatures, every transaction and account.
- **Wallet:** in-browser ML-DSA-44 keys, faucet (1,000 tQPF per hour), signed transfers, seed backup.
- **Live stream:** server-sent events push each new block as it is finalised.

```
GET  /api/stats
GET  /api/blocks?n=20
GET  /api/block/:height|hash
GET  /api/tx/:hash
GET  /api/account/:qpf1…
POST /api/tx
POST /api/faucet
GET  /api/live  (SSE)
```

Per-block cost

Each block carries four ML-DSA-65 signatures (13,236 bytes) and requires four verifications (~1.8 ms on one core). At 200 transactions per block, admission verification costs ~66 ms per slot, about 1.7% of a 4-second slot on a single core.

Roadmap

PHASE	SCOPE
v1 · Testnet (live)	ML-DSA blocks and transactions, explorer, wallet, faucet
v2 · Open validators	Staked validator set, BFT with ML-DSA votes, multi-node gossip
v3 · Programmability	Contract runtime with PQ-native account abstraction and multisig
v4 · Bridges	Hash-based attestations for assets moving in from ECC chains
v5 · Privacy	Lattice-based commitments for confidential transfers

Roadmap phases describe planned work and may change as research and audits progress.

$QPF and disclosures

$QPF

$QPF is the QuantumProof community token, launched on pump.fun on Solana. Its contract address is published on quantumproofchain.net with a copy button. **tQPF** is the testnet gas asset; it is a separate asset with no monetary value and cannot be exchanged for $QPF.

Risks and disclosures

- QuantumProof is in testnet. The validator set is permissioned and operated by the project.
- $QPF lives on Solana, which uses Ed25519. The token itself inherits Solana's security model; the post-quantum guarantees in this paper apply to the QuantumProof chain.
- ML-DSA is standardised but young. Cryptanalysis may advance, and parameters may need to change.
- Post-quantum signatures increase transaction and block sizes, with bandwidth and storage costs.
- The code has not yet been independently audited. Audits are planned before any mainnet.
- Nothing in this document is an offer of securities or a promise of returns.

References

- NIST FIPS 204, *Module-Lattice-Based Digital Signature Standard*, August 2024.
- NIST FIPS 205, *Stateless Hash-Based Digital Signature Standard*, August 2024.
- NIST FIPS 202, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, 2015.
- P. W. Shor, *Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer*, 1997.
- L. K. Grover, *A Fast Quantum Mechanical Algorithm for Database Search*, 1996.
- Ducas et al., *CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme*, 2018.
- RFC 8032, *Edwards-Curve Digital Signature Algorithm (EdDSA)*, 2017.

Move before Q-day. **quantumproofchain.net**